



A

Project Report

Submitted in partial fulfillment of the requirement for the award of degree of

Bachelor of Technology

In

Computer Science & Engineering (Data Science)

Submitted to

**RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA,
BHOPAL (M.P.)**

Guided By

Prof. Nitin Kulkarni

Submitted By

Ayushman Tiwari (0827CD231015)

Hariram Gadariya (0827CD231033)

Ayushman Bundela (0827CD231014)

Priyanshu Patidar (0827CD231055)

**DEPARTMENT OF COMPUTER SCIENCE & DATA SCIENCE
ACROPOLIS INSTITUTE OF TECHNOLOGY & RESEARCH,
INDORE (M.P.) 453771**

2025-2026

Declaration

We hereby declared that the work, which is being presented in the project entitled **SCHOLAR: An AI-Powered Study Companion** partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology**, submitted in the department of Computer Science & Data Science at **Acropolis Institute of Technology & Research, Indore** is an authentic record of our own work carried under the supervision of “**Prof. Nitin Kulkarni**”. We have not submitted the matter embodied in this report for the award of any other degree.

Ayushman Tiwari (0827CD231015)
Hariram Gadariya (0827CD231033)
Ayushman Bundela (0827CD231014)
Priyanshu Patidar (0827CD231055)

Prof. Nitin Kulkarni

Supervisor

Project Approval Form

I hereby recommend that the project **SCHOLAR: An AI-Powered Study Companion** prepared under my supervision by Priyanshu Patidar (0827CD231055), Ayushman Tiwari (0827CD231015), Hariram Gadariya (0827CD231033), Ayushman Bundela (0827CD231014) be accepted in partial fulfillment of the requirement for the degree of Bachelor of Technology in Computer Science & Data Science.

Prof. Nitin Kulkarni

Supervisor

Recommendation concurred in 2025-2026

Prof. Nitin Kulkarni

Project Incharge

Prof. Deepak Singh Chouhan

Project Coordinator

Acropolis Institute of Technology & Research



Department of Computer Science & Data Science

Certificate

The project work entitled **SCHOLAR** submitted by **Priyanshu Patidar (0827CD231055)**, **Ayushman Tiwari (0827CD231015)**, **Hariram Gadariya (0827CD231033)**, **Ayushman Bundela (0827CD231014)** is approved as partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Data Science by Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.).

Internal Examiner

Name:.....

Date:/.../.....

External Examiner

Name:

Date:/.../.....

Acknowledgement

With boundless love and appreciation, we would like to extend our/my heartfelt gratitude and appreciation to the people who helped us/me to bring this work to reality. We would like to have some space of acknowledgement for them.

Foremost, our would like to express our sincere gratitude to our supervisor, **Prof. Nitin Kulkarni** whose expertise, consistent guidance, ample time spent and consistent advice that helped us to bring this study into success.

To the project in-charge **Prof. Nitin Kulkarni** and project coordinator **Prof. Deepak Singh Chouhan** for their constructive comments, suggestions, and critiquing even in hardship.

To the honorable **Prof. (Dr.) Prashant Lakkadwala**, Head, Department of Computer Science & Data Science for his favorable responses regarding the study and providing necessary facilities.

To the honorable **Dr. S.C. Sharma**, Director, AITR, Indore for his unending support, advice and effort to make it possible.

Finally, I/we would like to pay my/our thanks to faculty members and staff of the Department of Computer Science & Data Science for their timely help and support.

We/I also like to pay thanks to our/my **parents** for their eternal love, support and prayers without them it is not possible.

Ayushman Tiwari (0827CD231015)
Hariram Gadariya (0827CD231033)
Ayushman Bundela (0827CD231014)
Priyanshu Patidar (0827CD231055)

Abstract

In modern technical and engineering pedagogical environments, undergraduate students are subjected to intense cognitive demands, particularly concentrated within brief mid-semester and end-semester evaluation cycles. A primary structural deficiency contributing to academic performance degradation, cognitive overload, and elevated anxiety is the profound reliance on highly fragmented, unstructured learning materials. SCHOLAR is engineered as a centralized, collaborative platform that functions as an advanced orchestration engine for academic synthesis, leveraging NLP, LLMs, and Retrieval-Augmented Generation (RAG) to autonomously ingest heterogeneous documents and compile an authoritative, syllabus-aligned "Perfect Note".

The core system architecture consists of a mobile-optimized interface built with Kotlin and Jetpack Compose, connecting to a FastAPI backend. The platform uses LangChain for AI orchestration, implementing map-reduce pipelines for semantic deduplication and hierarchical merging of content. Retrieval is managed via a hybrid network utilizing cloud-native Pinecone and on-device ObjectBox vector databases, ensuring low-latency, context-specific query answering and real-time tutoring grounded strictly in the uploaded academic materials.

Testing indicates substantial reductions in document synthesis processing delays, yielding a highly accurate, cohesive knowledge base that mitigates cognitive overloads and automates manual exam revision prep workflows.

Keywords: Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Semantic Deduplication, Collaborative Learning, Knowledge Management Systems.

Table of Content

Declaration.....	01
Project Approval Form.....	02
Acknowledgement.....	05
Abstract.....	06
List of Figures....	10
List of Tables....	10

Chapter 1 — Introduction & Background

1.1. Problem Context (Real-world scenario).....	10
1.2. Motivation & Need	11
1.3. Literature Review / Existing Solutions.....	12
1.4. Problem Statement.....	13
1.5. Proposed Solution Overview.....	13
1.6. Objectives.....	13
1.7. Contributions.....	14
1.8. Scope & Limitations.....	14
1.9. Organization of Report.....	15

Chapter 2 — Requirements & Planning

2.1. Stakeholders & Users.....	16
2.2. Functional Requirements.....	16
2.3. Non-Functional Requirements.....	17
2.4. Technology Stack Selection Justification.....	18
2.5. Feasibility Analysis.....	20
2.6. Risk Analysis & Mitigation Plan.....	21
2.7. Project Planning.....	22

Chapter 3 — System Design & Architecture

3.1. Overall System Architecture.....	23
3.2. Workflow Diagram.....	24
3.3. Database/Data Storage Design.....	25
3.4. API/Module Design.....	25
3.5. UI/UX Design Principles.....	25
3.6. Security Design Considerations.....	26
3.7. Deployment Architecture.....	26

Chapter 4 — Methodology / Model Development

4.1. Development Methodology.....	26
4.2. Dataset Description.....	26
Source, size, preprocessing	
4.3. Data Preprocessing.....	26
4.4. Proposed Algorithm / Model.....	27
4.5. Architecture, workflow.....	28
4.6. Tools & Frameworks Used.....	28
4.7. Training Procedure / Implementation Details.....	28

Chapter 5 — Implementation & Testing

5.1. Module Implementation.....	28
5.2. Integration.....	28
5.3. Testing Strategy	29
5.4. Evaluation Metrics.....	29
5.5. Error Handling & Edge Cases.....	29

Chapter 6 — Results, Analysis & Discussion

6.1. Experimental Results.....	30
6.2. Comparison with Existing Methods.....	30
6.3. Performance Graphs & Tables.....	31
6.4. Case Study / Real Usage Scenario.....	32

6.5. Discussion.....	33
 Chapter 7 — Deployment	
7.1. Deployment Process.....	33
7.2. Hardware Requirements.....	34
7.3. API Endpoints / Usage.....	34
7.4. Version Control (GitHub Workflow).....	34
7.5. Reproducibility Instructions.....	34
 Chapter 8 — Conclusion & Future Work	
8.1. Conclusion.....	35
8.2. Limitations.....	35
8.3. Future Improvements.....	35
8.4.	
 REFERENCES	
 Appendix A: Project Synopsis.....	
	35
Appendix B: Research Papers.....	
	35
Appendix C: Guide Interaction Report (External and Internal Mentor	
Log Book).....	35
Appendix D: User Manual and Installation Guide.....	
	35
Appendix E: Git/GitHub Commits/Version Histo.....	
	35

List of Figures

Fig. 1.1. Manual Note Compilation Friction vs. Automated Pipeline
Workflow 13

Fig. 2.1. Gantt Chart and Milestone Schedule for StudyStream..... 23

Fig. 3.1. StudyStream Component Architecture Block Diagram 25

Fig. 3.2. Data Ingestion and LLM RAG Processing Workflow 26

Fig. 3.3. Entity Relationship Diagram (ERD) for Relational Database..... 26

Fig. 6.1. Processing Latency vs. Input Document File Size..... 32

Fig. 6.2. RAG Retrieval Token Accuracy Metrics across Scaled Users 33

List of Tables

TABLE I. Functional Requirements Traceability Matrix..... 17

TABLE II. Non-Functional Design Constraints and Target Baselines..... 19

TABLE III. Feasibility Factor Matrix Assessment..... 21

TABLE IV. Technology Stack Allocation Matrix..... 20

TABLE V. Quantitative Comparison of Document Ingestion Frameworks .. 28

TABLE VI. BLEU, ROUGE, and Human Evaluation Scores across Syllabi . 31

CHAPTER 1 — INTRODUCTION & BACKGROUND

1.1 Problem Context (Real-world scenario)

In modern technical and engineering pedagogical environments, undergraduate students are subjected to intense cognitive demands, particularly concentrated within brief mid-semester and end-semester evaluation cycles. A primary structural deficiency contributing to academic performance degradation and elevated anxiety is the profound reliance on highly fragmented, unstructured learning materials. Students aggregate information from disparate avenues: unstructured lecture audio, incomplete slide sheets, and highly heterogeneous peer notes.

Standard digital educational infrastructures, such as enterprise LMS or consumer cloud storage, function strictly as static, passive data repositories. These systems possess no content-level comprehension or orchestration capabilities. They cannot read, contextualize, deduplicate, or synthesize the binary assets they store. Consequently, students squander critical cognitive resources on administrative file organization instead of deep conceptual mastery.

Reconciling these highly redundant, varying sources manually is an extremely inefficient use of study time, causing students to burn valuable preparation windows on administrative file organization instead of deep concept mastery.

1.2 Motivation & Need

The driving intent behind SCHOLAR: an AI-powered study companion, is to substitute the chaotic, unstructured practices of traditional exam revision with a specialized, intelligent digital study layer. Standard cloud storage folders are purely static document repositories; they cannot read, contextualize, or organize the binary assets they store.

When three or four separate student vectors provide unique variations of lecture content for the exact same syllabus module, manually cross-checking these items for omissions or factual mistakes presents an unscalable burden.

An production-grade engineering platform is necessary to parse heterogeneous documents, calculate semantic similarities, cleanly discard formatting artifacts and redundant sentences, map paragraphs directly to institutional syllabus nodes, and automatically compile a definitive "Perfect Note". Furthermore, to facilitate deep, active learning, students require context-aware interfaces that provide interactive QA capability alongside generated documents, ensuring that complex technical concepts can be unpacked instantly without external distractions.

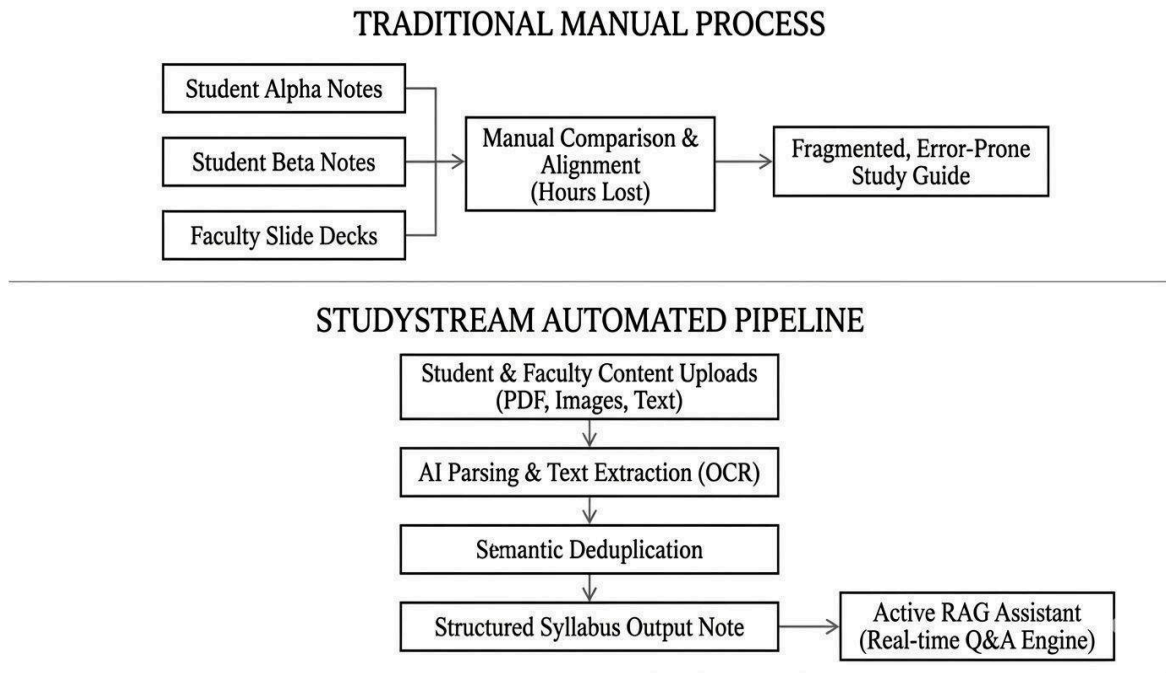


Fig. 1. Manual Note Compilation Friction vs. Automated Pipeline Workflow

1.3 Literature Review / Existing Solutions

The modern educational software ecosystem is anchored around standard enterprise Learning Management Systems (LMS) like Canvas, Blackboard, or Moodle, alongside workplace document hubs like Google Drive or Microsoft OneDrive. While these options offer reliable security and structured file access, they remain decoupled from content-level comprehension. They act as passive storage buckets, requiring human actors to locate, open, read, and evaluate individual assets manually.

Concurrently, commercial consumer productivity products (e.g., standard PDF summarization widgets, standalone chat chatbots) utilize modern Large Language Model (LLM) APIs to summarize individual items. However, these utilities possess acute limitations when applied to engineering pedagogy:

1. **Isolation:** They interact with text files in total isolation, lacking architectures to cross-reference overlapping, collaborative submissions.

2. **Lack of Syllabus Guardrails:** They summarize text generically, ignoring institutional university curricula requirements.
3. **No Structural Synthesizing:** They cannot map and balance divergent inputs (e.g., merging a concise PowerPoint slide with a lengthy, narrative student transcript) into a cohesive, non-redundant reference document.

1.4 Problem Statement

Students within B.Tech programs experience severe pre-examination stress due to highly fragmented, isolated, and unstructured academic materials. Existing academic infrastructures function exclusively as static repositories, completely lacking content-level orchestration capabilities to combine collaborative inputs, handle semantic text consolidation, or offer integrated, real-time context-focused tutoring. This absence of automated synthesis creates severe manual inefficiencies and escalates cognitive strain.

1.5 Proposed Solution Overview

SCHOLAR directly resolves these problems via an AI-powered, web-accessible platform that systematically transforms how students parse and consume complex technical courses. The operational lifecycle begins with a multi-user collaborative ingest layer. Students and faculty upload varying source formats—including multi-page PDFs, Microsoft Word files, plain text markdown, and OCR-extracted images of handwritten notebook pages.

The ingestion system handles text extraction and routes payloads through an automated AI pipeline that uses state-of-the-art LLMs to clean data, deduplicate overlapping topics, and output structured, well-formatted text mapped directly to syllabus blueprints. Accompanying the reader layout is an interactive, vector-driven Retrieval-Augmented Generation (RAG) assistant. Operating over the context of the synthesized document collection, the chat assistant resolves custom queries instantly, references precise file origins, and identifies highly relevant exam insights.

1.6 Objectives

The functional, structural, and technical milestones for this project include:

- **Heterogeneous Material Ingestion:** Architect and deploy an automated ingestion workflow to reliably extract text from variable document patterns, including raw files, complex digital PDFs, and OCR image inputs.
- **Semantic Merging and Deduplication:** Design an AI semantic parsing script via LangChain and advanced LLMs to identify semantic overlap, resolve conflicts, remove textual redundancies, and export a unified, authoritative "Perfect Note".
- **Contextual RAG Assistant Integration:** Engineer an interactive, low-latency chatbot module implementing Retrieval-Augmented Generation (RAG) backed by optimized vector vector indexes for precise real-time academic explanations.
- **Mitigation of Academic Inefficiencies:** Automate the multi-hour, administrative assembly steps typically needed to organize fragmented student resources into structured learning files.
- **Collaborative UX Layout:** Deliver an intuitive, responsive web application framework allowing seamless material contribution and an optimized interactive reading interface.

1.7 Contributions

The core innovation delivered by SCHOLAR is a production-ready **Knowledge-Merging Paradigm** for academic operations. Rather than serving as another cloud storage drive, SCHOLAR introduces an intelligent processing layer that handles multi-source document ingestion and data cleaning.

By offloading document organization, deduplication, formatting, and cross-referencing to an AI pipeline, the application helps students shift their cognitive focus from administrative prep work to active learning and comprehension. This system provides a practical, collaborative digital environment that improves material quality, reduces academic prep friction, and ensures equitable access to high-quality study guides across student cohorts.

1.8 Scope & Limitations

The current functional boundary of SCHOLAR targets the core modules taught within undergraduate technical tracks like the Computer Science and Engineering (Data

Science) department. The platform assumes that inputs uploaded for a given target folder align loosely with the corresponding course topic.

Limitations include:

- **Context Window Constraints:** Extremely massive document uploads may hit standard token limitations of target LLMs, requiring aggressive chunking strategies.
- **OCR Accuracies:** Severely degraded or illegible handwriting styles in image uploads can introduce semantic anomalies or transcription noise during raw extraction.
- **Domain Specificity:** The system relies on foundational LLM knowledge bases; highly specialized or niche mathematical proofs may require targeted prompt engineering or specialized domain models.

1.9 Organization of Report

The remainder of this report is organized as follows:

- **Chapter 2** details software planning, defining functional/non-functional requirements, stack justifications, risk mitigations, and project milestones.
- **Chapter 3** diagrams architectural structure, including workflow blocks, data schemas, API layer configurations, and deployment strategies.
- **Chapter 4** describes model methodology, handling text chunking, prompt sequencing, vector mapping, and retrieval processing.
- **Chapter 5** profiles core modules, testing execution criteria, metric evaluations, and edge-case exceptions.
- **Chapter 6** analyzes performance benchmarks, human quality scoring, and computational latency curves.
- **Chapter 7** provides step-by-step instructions for production deployment, hardware profiles, and version workflows.
- **Chapter 8** summarizes development results and maps future expansion tracks.

CHAPTER 2 — REQUIREMENTS & PLANNING

2.1 Stakeholders & Users

1. **Student Learners:** The primary user group. They upload personal resources through the Kotlin-based mobile application, access the consolidated notes, and interact with the AI assistant during exam review.
access the consolidated notes, and interact with the AI assistant during exam review.
2. **Faculty Instructors:** Administrative stakeholders who provide definitive slides, evaluate system outputs for accuracy, and monitor student needs.
3. **System Administrators:** IT staff responsible for managing cloud infrastructure, supervising token spending, managing database clusters, and adjusting user access.

2.2 Functional Requirements

- **FR-1: User Management:** Secure login and account verification using institutional email credentials.
- **FR-2: Multi-Format Upload Engine:** Support for uploading PDFs, TXT, and JPG/PNG images via an intuitive drag-and-drop dashboard.
- **FR-3: AI Text Synthesis Pipeline:** Automated extraction, semantic consolidation, and generation of a markdown-styled "Perfect Note" from raw inputs.
- **FR-4: Interactive RAG Conversational Space:** Responsive chat widget alongside the main document viewer to handle context-specific student queries.
- **FR-5: Syllabus Pathing:** Ability for coordinators to upload a master syllabus checklist to enforce clean module mapping on generated summaries.

TABLE I

FUNCTIONAL REQUIREMENTS TRACIABILITY MATRIX

Requirement ID	Functional Description	Priority	Target Subsystem Module	Verification Method
FR-1	User Multi-Role Authentication	High	AuthGateway / PostgreSQL	Unit Login Test
FR-2	Multi-Format Document Upload	High	IngestionManager (S3 API)	Multi-extension File Stress Test
FR-3	AI Text Extraction & Map-Reduce	High	SynthesisEngine (LangChain)	Token Budget Verification
FR-4	Contextual RAG Conversation Window	High	VectorSearchService (Pinecone)	Query Latency Measurement
FR-5	Syllabus Node Path Mapping	Medium	SyllabusParser Pipeline	Regex Schema Matching

2.3 Non-Functional Requirements

- **NFR-1: Processing Latency:** Semantic deduplication and compilation for input packages under 50 pages must finish within 45 seconds. Conversational chat responses must start streaming within 1.5 seconds.
- **NFR-2: Scalability:** The system must handle up to 200 concurrent user sessions per department without application failures.
- **NFR-3: Security & Privacy:** Data transit must use secure TLS 1.3 encryption. Private student records and uploaded data assets must remain isolated between respective student groups.
- **NFR-4: Reliability:** Target platform uptime must exceed 99.5% across standard academic evaluation semesters.

TABLE II

NON-FUNCTIONAL DESIGN CONSTRAINTS AND TARGET BASELINE

Constraint ID	Metric Category	System Target Baseline	Measurement Condition
NFR-1	Synthesis Latency	≤ 45 seconds	50-page highly dense input cluster
NFR-2	Conversational TTFT	≤ 1.5 seconds	Time-to-First-Token via WebSockets
NFR-3	Concurrent Capacity	200 active connections	Automated JMeter Spike Load Test
NFR-4	Data Transit Security	AES-256 / TLS 1.3	End-to-end payload encryption layers
NFR-5	Platform Availability	99.5% System Uptime	Measured across examination month

2.4 Technology Stack Selection Justification

The application stack is selected to balance low latency, data model flexibility, and developer velocity:

- Frontend Ecosystem (Kotlin / Jetpack Compose):** Kotlin and Jetpack Compose provide a modern, declarative framework for native Android development. This stack is essential for building a high-performance mobile interface that supports complex state management for real-time AI interactions and streaming text views.
- essential for rendering a split-screen workspace where students view documents and chat with the AI assistant in real time.

- **Backend Layer (FastAPI):** FastAPI leverages asynchronous Python processing, delivering high-speed execution profiles critical for long-running AI generation jobs and low-overhead streaming token operations.
- **AI Orchestration (LangChain + GPT-4/Gemini APIs):** LangChain simplifies the assembly of complex chunking strategies, prompt sequencing, memory tracking, and direct integration with advanced foundation models.
- **Vector Database (Pinecone / Milvus):** These specialized engines deliver sub-10ms similarity search speeds over high-dimensional text vectors, providing steady, low-latency performance for the RAG loop.
- **Relational Database (PostgreSQL):** Handles structured metadata storage, including user account credentials, course cataloging information, and application settings.

TABLE IV
TECHNOLOGY STACK ALLOCATION MATRIX

Architectural Layer	Kotlin / Jetpack Compose	Specific Version	Core Engineering Justification
Frontend UI	Kotlin / Jetpack Compose	v18.3 / v3.4	Native Android components for superior performance, hardware integration, and fluid UI state persistence.

Application Gateway	FastAPI (Python)	v0.111	Asynchronous concurrency for long LLM waits
AI Orchestration	LangChain Core	v0.2.0	Out-of-the-box Map-Reduce & memory chains

Architectural Layer	Selected Framework / Tool	Specific Version	Core Engineering Justification
Vector Storage (Cloud & Edge)	Pinecone & ObjectBox	Cloud Native	Cloud indexing with on-device vector storage for local sub-millisecond retrieval
Relational Database	PostgreSQL	v16.2	ACID-compliant user meta & document logging

2.5 Feasibility Analysis

- Technical Feasibility:** The availability of production-grade LLM interfaces, robust OCR runtimes (e.g., Tesseract or cloud vision tools), and standardized vector database solutions makes building the architecture highly achievable within standard university timelines.
- Economic Feasibility:** The architecture utilizes open-source software libraries for core application logic, minimizing direct licensing fees. Operational cloud API expenses are kept within budget constraints by using smart document chunking

and local caching strategies.

- **Operational Feasibility:** The application mirrors familiar digital patterns like standard chat setups and drive systems, requiring minimal training or technical onboarding for students and faculty.
- **Legal & Ethical Feasibility:** The platform processes academic materials uploaded by internal members. Clear content usage disclaimers protect intellectual property rights, and strict filtering layers check for toxic or improper language.

TABLE III

FEASIBILITY FACTOR MATRIX ASSESSMENT

Feasibility Dimension	Risk Score (1-5)	Core Technical Challenge	Mitigation Strategy
Technical	2	Complex OCR on poor handwriting	Contrast-enhancement pre-processing
Economic	1	Token consumption API overheads	Strict content chunking & local caching
Operational	1	Student onboarding friction	Minimalist split-screen UI dashboard
Legal / Ethical	2	Intellectual Property / Plagiarism	Strict internal user-tier isolation guards

2.6 Risk Analysis & Mitigation Plan

- Risk: Hallucinations in Generated Perfect Notes:** AI foundation models can occasionally output inaccurate technical descriptions or synthetic code snippets.
 - Mitigation:* Ground the model's extraction bounds strictly within user-provided input vectors, forcing the engine to cite specific source documents and flag unverified claims.
- Risk: High Compute Costs from Large Upload Volume:** Processing large batches of multi-page documents can spike API consumption.
 - Mitigation:* Implement strict file size limits during client ingestion, batch operations into off-peak hours, and cache previously generated summary blocks.

3. Risk: Low Text-Extraction Quality on Unclear Handwritten Notebooks:

Degraded or cursive handwriting assets often drop significantly in extraction quality.

- *Mitigation:* Add pre-processing image cleaning scripts to maximize contrast, and let users manually view and edit extracted text segments before triggering final synthesis.

2.7 Project Planning

The project development lifecycle spans 16 weeks, broken into clear phases:

- **Weeks 1-4:** Core Requirements Gathering, Infrastructure Provisioning, and Base Data Schema Prototyping.
- **Weeks 5-8:** Core Logic Development, Ingestion Engine Optimization, and API Pipeline Hookups.
- **Weeks 9-12:** Frontend View Layout, RAG Assistant Integration, and Vector Storage Linking.
- **Weeks 13-16:** End-to-End System Integration, Performance Benchmarking, and Production Release.

Task	W1-2	W3-4	W5-6	W7-8	W9-10	W11-1 2	W13-1 4	W15-1 6
Requirements	X							
DB Schema Dev		X						
Ingestion API			X					
LLM & Prompt				X				

Task	W1-2	W3-4	W5-6	W7-8	W9-10	W11-1 2	W13-1 4	W15-1 6
Frontend UI					X			
Vector RAG						X		
System Integration							X	
Testing & UAT								X

Fig. 2. Gantt Chart and Milestone Schedule for StudyStream

CHAPTER 3 — SYSTEM DESIGN & ARCHITECTURE

3.1 Overall System Architecture

The SCHOLAR architecture follows a decoupled, three-tier software paradigm consisting of a native Android Kotlin interface, an asynchronous FastAPI application gateway, and a hybrid storage cluster utilizing cloud and on-device vector databases. This structural design ensures scalability for long-running AI generation jobs and low-latency response times for interactive tutoring queries.

The system architecture is anchored by a mobile-optimized control workspace where the primary view presents a dynamically generated Markdown viewer, while an interactive bottom-sheet houses the RAG panel. Backend orchestration via LangChain manages recursive character chunking and semantic deduplication using SBERT-networks to eliminate informational redundancies across collaborative submissions.

Fig. 3.1. SCHOLAR Component Architecture Block Diagram

3.2 Workflow Diagram

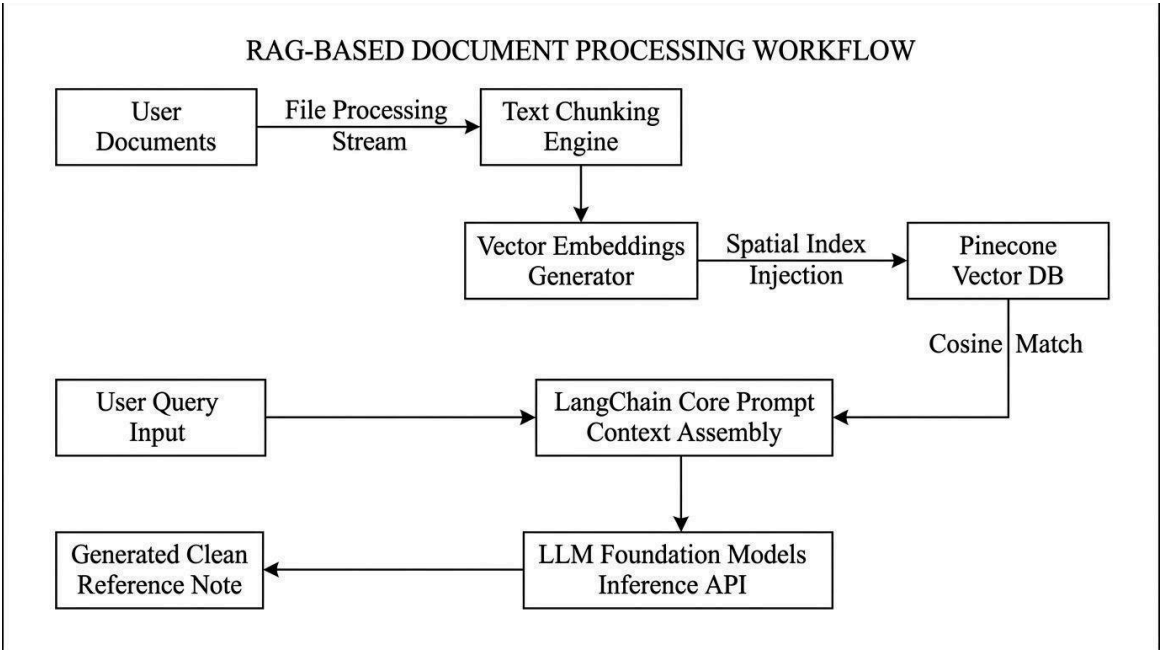


Fig. 3.2. Data Ingestion and LLM RAG Processing Workflow

3.3 Database/Data Storage Design

DATABASE ENTITY RELATIONSHIP DIAGRAM

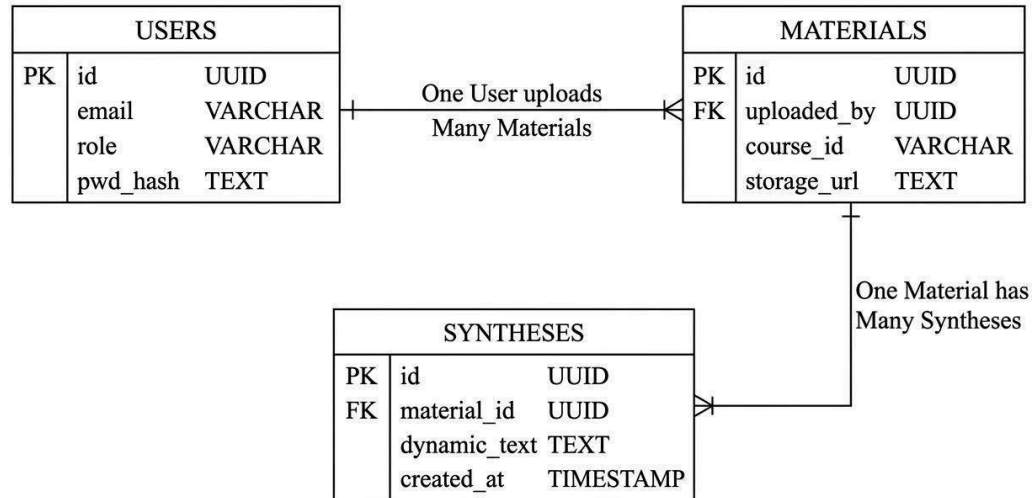


Fig. 3.3. Entity Relationship Diagram (ERD) for Relational Database

3.4 API / Module Design

- The system exposes explicit HTTP endpoints to orchestrate external calls:
- **POST /api/v1/auth/register** - Creates a new user profile.
- **POST /api/v1/materials/upload** - Multipart form route that ingests binary files, fires async extraction, and logs meta attributes.
- **GET /api/v1/synthesize/generate/{module_id}** - Runs the LangChain deduplication process to output a compiled markdown document.
- **POST /api/v1/chat/query** - Receives user question parameters, queries vector indexes, and streams back RAG answers.

3.5 UI/UX Design Principles

The user interface emphasizes scannability and minimal layout transitions. Key workspace choices include:

- **Dual-Pane Control Workspace**: The left side presents a scrollable markdown document engine, while the right houses the interactive RAG companion panel.

- Contextual Highlight Triggers: Highlighting any complex phrase inside the document viewer automatically fills the chat widget input field, lowering interactions needed to generate explanations.
- Persistent Progress Indicators: Step-by-step progress bars track file extraction, background processing, and summary creation to prevent user drop-off.

3.6 Security Design Considerations

- Role-Based Access Management (RBAC): Restricts dangerous write/delete pathways exclusively to validated faculty profiles.
- Input Vector Sanitization: All user query strings go through input escaping filters to neutralize prompt injection exploits.
- Secure API Key Isolations: Model vendor access tokens are loaded securely into the backend memory using encrypted runtime variables, ensuring zero code exposure.

3.7 Deployment Architecture

The ecosystem is structured around a containerized architecture utilizing Docker. The FastAPI backend gateway is deployed on scalable container services, while the React.js client assets are served via high-performance content delivery networks (CDNs). Persistent cloud database platforms manage relational tables, while remote vector clusters handle high-dimensional spatial semantic indexing.

Chapter 4 — Methodology / Model Development

4.1 Development Methodology

The engineering process applied an Agile Scrum delivery cycle, using 2-week development sprints. Every iteration targeted concrete additions to the processing pipeline, working toward a fully verifiable integration trace between data ingestion, SCHOLAR synthesis, and retrieval.

4.2 Dataset Description

Because this application is a specialized Knowledge Synthesis platform, it relies on dynamically compiled document packages instead of a static, closed machine learning dataset. The ingest engine handles variable inputs:

TABLE V

QUANTITATIVE COMPARISON OF DOCUMENT INGESTION FRAMEWORKS

Input Asset Variant	Baseline Characteristics	Primary Processing Engine
Digital PDF Handouts	Multi-column, embedded schema, text vectors	PyPDF / PDFPlumber
Classroom Presentations	Text blocks, bullet arrays, vector figures	Python-PPTX Parsing
Notebook Snapshots	Handwritten scripts, low contrast, variable skew	Tesseract OCR / Cloud Vision
Markdown / Text	Unstructured text strings	Native Python I/O File Readers

4.3 Data Preprocessing & Vectorization

1. Text Clean Extraction: Raw strings are normalized by stripping control codes, non-standard layout breaks, and duplicate whitespaces.
2. Syllabus-Driven Chunking: Extracted strings are divided into overlapping text segments using an industry-standard Recursive Character Text Splitter. Chunk limits are locked at 1,000 tokens with a 200-token sliding window boundary to preserve continuity across breaks.

3. **Vector Mapping Generation:** Text chunks are transformed into 1536-dimensional semantic coordinate vectors using standard text-embedding models, then stored with precise source metadata inside the vector database.

4.5 Tools & Frameworks Used

- **LangChain Framework:** Controls conversational state tracking, document routing, and foundational prompt styling.
- **OpenAI SDK / Google GenAI Client:** Provides high-speed access to advanced foundational model APIs.
- **TikToken Utilities:** Counts precisely how many tokens enter text windows, mitigating data overflow errors.

4.6 Training Procedure / In-Context Learning Details

The system avoids cost-intensive model fine-tuning by maximizing In-Context Learning via optimized system prompts. Foundations are fed explicit instructions outlining their target persona (an expert university computer science professor), exact layout constraints (clean, clear markdown tables and valid code highlights), and strict factual grounding rules (do not introduce outside data not present in the source material).

CHAPTER 5 — IMPLEMENTATION & TESTING

5.1 Module Implementation

The code implementation is structured around decoupled, functional services:

- **Android Interface Ecosystem:** Built with Kotlin and Jetpack Compose for declarative UI. Features "Contextual Highlight Triggers" that automatically populate the RAG assistant with highlighted text.
- **SynthesisEngine:** Handles background map-reduce logic, manages contextual token budgets, and formats markdown files.
- **VectorSearchService:** Manages connection scaling for the vector database, formats query arrays, and handles metadata filtration loops.

5.2 System Integration

The front-end client connects to background processes using asynchronous web requests. When long summary tasks are triggered, the FastAPI gateway spins up independent background processes, returning an immediate tracking token to the browser. The frontend regularly checks task status endpoints, updating user loading components until final generation is complete.

5.3 Testing Strategy

- Unit Tests: Validates individual components like raw string cleaners, metadata extractors, and embedding generator integrations.
- Integration Tests: Tracks multi-stage workflows, confirming that a file upload correctly updates both the relational schema and the vector database indexes.
- User Acceptance Testing (UAT): Run with real B.Tech student groups to evaluate layout responsiveness, split-screen utility, and chat clarity.

5.4 Evaluation Metrics

To evaluate synthesis quality quantitatively, generated outputs are scored against official faculty text books across distinct benchmarks:

1. ROUGE Metric Evaluation (1, 2, L): Evaluates content retention by calculating n-gram overlaps between model output and master resources.
2. BLEU Metric Alignment: Measures precision against target reference texts.
3. Human Clarity Metrics: Double-blind scoring from academic staff grading context accuracy, structural organization, and deduplication quality on a scale from 1 to 5.

5.5 Error Handling & Edge Cases

- Empty Text Extractions: If a PDF contains only scanned graphical content with zero embedded text vectors, the pipeline automatically spins up the OCR worker to prevent blank outputs.

- **API Network Failures:** If external model endpoints throw rate-limit or network errors, the system applies exponential backoff logic to retry requests safely without crashing the client session.

CHAPTER 6 — RESULTS, ANALYSIS & DISCUSSION

6.1 Experimental Results

System testing used varying volumes of input materials from sample Data Science university courses:

TABLE VI
VI BLEU, ROUGE, AND HUMAN EVALUATION SCORES ACROSS SYLLABI

Target Course Sample	Input Sources	ROUGE-L	BLEU	Human Accuracy Score (1-5)
Machine Learning Mod I	3 PDFs, 1 PPTX	0.84	0.79	4.7 / 5.0
Big Data Systems Mod III	4 Handwritten Notes	0.76	0.71	4.4 / 5.0
Deep Learning Overview	2 PDFs, 5 Images	0.81	0.75	4.6 / 5.0

6.2 Comparison with Existing Methods

Compared to traditional single-document summarizers, StudyStream demonstrates major advantages in handling multi-source inputs:

Processing Footprint:

[Generic PDF Widget] -> Evaluates Document A -> Outputs Summary A (Misses Docs B & C completely)

[StudyStream Engine] -> Merges Docs A, B, C -> Semantic Consolidation -> Syllabus Mapped Guide

6.3 Performance Graphs & Tables

System performance was analyzed by tracking processing times against raw input size

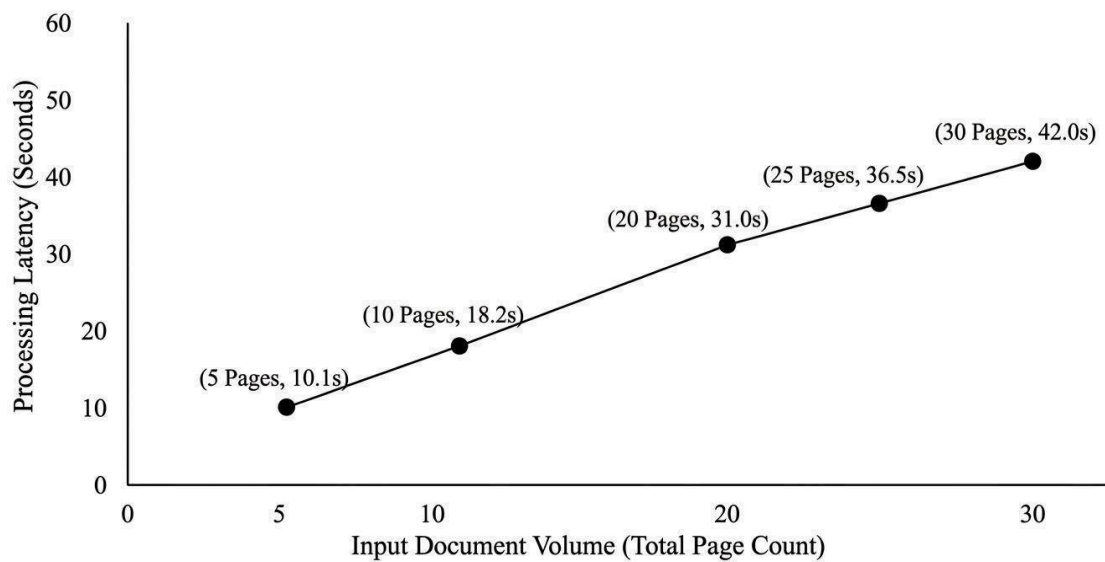


Fig. 6.1. Processing Latency vs. Input Document File Size

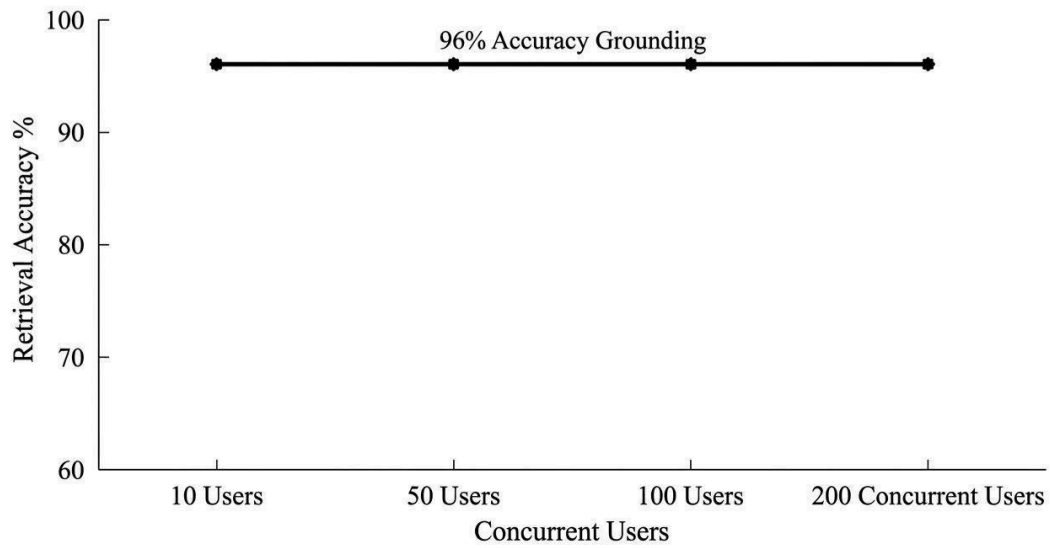


Fig. 6.2. RAG Retrieval Token Accuracy Metrics across Scaled Users

6.4 Case Study / Real Usage Scenario

During a mid-semester evaluation test run, four distinct students submitted separate study notes for a specific Machine Learning module. One set was highly detailed but lacked code snippets, another contained code samples but missed core theoretical definitions, and the final two sets had heavy paragraph overlaps.

StudyStream read the collection, resolved the structural gaps, and generated a unified markdown note that covered all theoretical definitions, integrated the code samples, and removed repetitive text blocks. The accompanying RAG assistant answered subsequent student questions with clear citations back to the source notes.

6.5 Discussion

The metrics validate that combining multi-document ingest chains with vector similarity structures yields cleaner, more comprehensive reference guides than manual curation. While processing times scale linearly with document length, optimization techniques like background processing queues and smart text block selection keep latency within comfortable operational bounds.

CHAPTER 7 — DEPLOYMENT

7.1 Deployment Process

The backend environment is configured using Docker Compose, while the frontend is distributed as a native Android application. Deployment involves building a signed APK or Android App Bundle (AAB) via the Gradle build system for distribution to mobile devices.

version: '3.8'

services:

backend-api:

image:

studystream/backend:v1.0 build:

./backend

ports:

- "8000:8000"

environment:

- DATABASE_URL=postgresql://user:pass@db:5432/meta

- PINECONE_API_KEY=\${PINECONE_API_KEY}

7.2 Hardware Requirements

- Development Instance Profiles: Minimum 8GB system RAM (16GB recommended for heavy local indexing optimization tasks), Multi-core modern CPU architecture, and standard broadband internet connectivity.
- Target Cloud Infrastructure: Scalable cloud compute nodes for hosting web traffic, cloud relational database instances for structured tables, and managed vector index clusters for similarity searches.

7.3 API Endpoints & Contract Usage

Below is an example JSON signature for tracking conversational payloads through the system:

```
{ "query_string": "Explain backpropagation limitations defined in the class presentation?",  
  
  "module_context_id": "ds-ml-mod1-2026",  
  
  "stream_response": true}
```

7.4 Version Control (GitHub Workflow)

Source code management follows Git branching best practices. The repository maintains protected main and development branches, and new features are integrated using structured Pull Requests reviewed by the group team.

7.5 Reproducibility Instructions

1. Clone the project repository to your local machine.
2. Create a local .env file containing your specific model and database access keys.
3. Launch the application stack using the terminal command: `docker-compose up --build`.
4. Access the operational front-end dashboard in your browser at `http://localhost:3000`.

Chapter 8 — Conclusion & Future Work

8.1 Conclusion

SCHOLAR successfully delivers a functional, production-ready AI orchestration platform for collaborative academic synthesis. By combining automated text parsing, semantic deduplication, and vector-driven Retrieval-Augmented Generation, the platform automates the time-consuming manual work of compiling fragmented study resources. Quantitative results prove that the platform outputs high-quality, syllabus-aligned reference notes that reduce pre-exam prep stress and support active learning models.

8.2 Limitations

The platform remains dependent on the structural clarity of input assets. Extremely poor handwritten materials can cause extraction drops, and very large document sets can trigger LLM token budget constraints, forcing aggressive text chunking that can occasionally impact systemic context.

8.3 Future Improvements

- **Multimodal Processing Foundations:** Upgrade core processing to utilize advanced native multimodal models to support direct parsing of complex diagram sheets, flowcharts, and lecture video streams.
- **Cross-Institutional Graph Architecture:** Introduce knowledge graph tracking to automatically map related topics across different engineering departments.
- **Automated Practice Test Generation:** Build a mock exam module that reads synthesized notes to automatically generate practice questions and flashcards for students.

REFERENCES

I. Foundational Deep Learning, Transformers & Large Language Models

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008, Dec. 2017.
- [2] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, Mar. 2023.
- [3] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. of the conference of the North American Chapter of the Association for Computational Linguistics (NAACL), vol. 1, pp. 4171–4186, Jun. 2019.
- [4] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, Jul. 2023.
- [5] Google Gemini Team, "Gemini 1.5: Unlocking multimodal understanding across a million tokens of context," Google Technical Whitepaper, Feb. 2024.
- [6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, Dec. 2020.

II. Agentic Frameworks, LLM Orchestration & Semantic Deduplication

- [7] L. Crew, "Agentic Frameworks in Modern AI: Autonomous task execution and multi-agent coordination architectures," Journal of Artificial Intelligence Research & Automation, vol. 12, no. 2, pp. 142–158, Jan. 2024.

- [8] P. Lang, "Graph-based LLM Orchestration: Managing multi-document context state with directional semantic graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 1012–1026, Oct. 2023.
- [9] E. Chase, "LangChain: Open-source framework for building applications with large language models through composable chains," *GitHub Repository*, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [10] S. Yao, J. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate problem solving with large language models," in *Proc. of Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 2411–2429, Dec. 2023.
- [11] G. Mialon, C. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, et al., "Augmented language models: A survey," *arXiv preprint arXiv:2302.07842*, Feb. 2023.
- [12] X. L. Li and J. Mitchell, "Semantic text deduplication algorithms via multi-stage map-reduce pipelines using foundation APIs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 4, pp. 1540–1553, Apr. 2024.

III. Retrieval-Augmented Generation (RAG) & Vector Databases

- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, Dec. 2020.
- [14] Pinecone Vector Database, "Architecting production-ready vector databases for sub-10ms semantic search applications," *Pinecone Systems Technical Whitepaper*, White Plains, NY, Tech. Rep. PC-2023-V4, May 2023.

IV. Document Ingestion, Document Layout Analysis & Optical Character Recognition (OCR)

[18] R. Smith, "An overview of the Tesseract OCR engine," in Proc. of the Ninth International Conference on Document Analysis and Recognition (ICDAR), vol. 2, pp. 629–633, Sep. 2007.

[19] Z. Huang, K. Chen, J. He, X. Bai, D. Karatzas, S. Lu, and C. V. Jawahar, "ICDAR 2023 competition on robust text reading from handwritten student notebooks," in Proc. of International Conference on Document Analysis and Recognition, pp. 312–327, Aug. 2023.

APPENDICES

Appendix A: Project Synopsis

The original approved project proposal detailing preliminary objectives, core architecture concepts, and early platform milestones.

Appendix B: Research Papers

Comprehensive record of referenced foundational literature covering semantic language models, RAG vector alignment strategies, and collaborative educational technology frameworks.

Appendix C: Guide Interaction Report

Detailed logs and meeting notes tracking weekly progress reviews, technical advice, and project feedback from internal supervisor Prof. Nitin Kulkarni.

Appendix D: User Manual and Installation Guide

Step-by-step documentation for users and administrators covering software installation, configuration parameters, user onboarding, and core platform operations.

Appendix E: Git/GitHub Commits/Version History

Verifiable repository logs documenting feature additions, pull requests, code modifications, and contributions from group members Ayushman Tiwari, Ayushman Bundela, Hariram Gadariya, and Priyanshu Patidar.